

Practical Set Pandas (I)

Page No.:

Date / /

```
import pandas as pd
```

Series: A pandas 1-dimensional labelled array that can hold any data type. Think of it like a single column in a spreadsheet (1-dimensional)

```
data = [100, 102, 104]
```

```
series = pd.Series(data)
```

```
print(series)
```

```
# 0 100
```

```
1 102
```

```
2 104
```

Defining the index

```
series = pd.Series(data, index=['a', 'b', 'c']) # By adding index parameter
```

```
print(series)
```

```
# a 100
```

```
b 102
```

```
c 104
```

Accessing the index using .loc operator

```
print(series.loc['a'])
```

```
# 100
```

```
print(series.loc['b'])
```

```
# 102
```

```
print(series.loc['c'])
```

```
# 104
```

Changing the values by index

```
series.loc['a'] = 125
```

```
series.loc['b'] = 150
```

```
series.loc['c'] = 200
```

```
print(series)
```

```
# a 125
```

```
b 150
```

```
c 200
```

Note: .Series & .Dataframe is constructor/class not a function

Locate the value by integer position by .iloc operator

```
data = [100, 102, 104]
```

```
Series = pd.Series(data, index=['a', 'b', 'c'])
```

```
Print(Series.iloc[0]) # 100
```

```
Print(Series.iloc[1]) # 102
```

```
Print(Series.iloc[2]) # 104
```

Filtering the values

```
data = [100, 102, 104, 200, 202]
```

```
Series = pd.Series(data, index=['a', 'b', 'c', 'd', 'e'])
```

(a) Print(Series[Series >= 200]) # Greater than or equal to 200

(b) Print(Series[Series < 200]) # Less than 200

(a) # d 200

e 202

(b) # a 100

b 102

c 104

With dictionary

```
calories = {
```

```
    'Day 1': 1750,
```

```
    'Day 2': 2000,
```

```
    'Day 3': 2200
```

```
}
```

```
Series = pd.Series(calories)
```

```
Print(Series)
```

Day 1 1750

Day 2 2000

Day 3 2200

Accessing the dictionary Key wise

```
Print(Series.loc['Day 1']) # 1750
Print(Series.loc['Day 2']) # 2000
Print(Series.loc['Day 3']) # 2200
```

Updating the value in dictionary

```
Series.loc['Day 1'] = 2000
Series.loc['Day 2'] = 2200
Series.loc['Day 3'] = 2400
Print(Series)
```

Day	Value
Day 1	2000
Day 2	2200
Day 3	2400

Adding the value in dictionary Keys

```
Calories = {"Day 1": 1750, "Day 2": 2000, "Day 3": 2200}
Calories = pd.Series(Calories)
Calories['Day 1'] += 500 # 1750 + 500 = 2250 # With Dictionary Key
Calories.loc['Day 2'] += 500 # 2000 + 500 = 2500
Calories.iloc[2] += 500 # 2200 + 500 = 2700
Print(Calories)
```

Day	Value
Day 1	2250
Day 2	2500
Day 3	2700

Filtering the values

```
Calories = {"Day 1": 1750, "Day 2": 2100, "Day 3": 1700}
Calories = pd.Series(Calories)
Print(Calories[Calories >= 2000]) # Day 2 2100
Print(Calories[Calories < 2000]) # Day 1 1750
# Day 3 1700
```

Exercise: Create a series with 4 methods to get

the following output:

```

1 Bulbasaur
2 Ivysaur
3 Venusaur
4 Charmander
5 Charmeleon
6 Charizard

```

Method 1

```

Pokemon = ['Bulbasaur', 'Ivysaur', 'Venusaur', 'Charmander',
            'Charmeleon', 'Charizard']
index = [1, 2, 3, 4, 5, 6]
Pokemon = pd.Series(Pokemon, index=index)
print(Pokemon)

```

Method 2

```

Pokemon = ['Bulbasaur', 'Ivysaur', 'Venusaur', 'Charmander',
            'Charmeleon', 'Charizard']
Pokemon = pd.Series(Pokemon, index=[1, 2, 3, 4, 5, 6])
print(Pokemon)

```

Method 3

```

Pokemon = ['Bulbasaur', 'Ivysaur', 'Venusaur', 'Charmander',
            'Charmeleon', 'Charizard']
Pokemon = pd.Series(Pokemon)
Pokemon.index = [1, 2, 3, 4, 5, 6]

```



```
Print(Pokemon)
```

Method 4 : With Dictionary

```
Pokemon = { "1": 'Bulbasaur',
             '2': 'Ivysaur',
             '3': 'Venusaur',
             '4': 'Charmander',
             '5': 'Charmeleon',
             '6': 'Charizard'
           }
```

```
Pokemon = pd.Series(Pokemon)
Print(Pokemon)
```

DataFrame: A tabular data structure with rows and columns (2-dimensional). Similar to excel spreadsheet.

```
data = { "Name": ["Spongebob", "Patrick", "Squidward"],
         "Age": [30, 35, 50] }
```

```
data = pd.DataFrame(data, index=["Employee 1", "Employee 2",
                                "Employee 3"])
```

```
Print(data)
```

#	Name	Age
Employee 1	Spongebob	30
Employee 2	Patrick	35
Employee 3	Squidward	50

Accessing the dataframe by .loc function


```
print(data.loc['Employee 1'])
```

```
print(data.loc['Employee 2'])
```

```
print(data.loc['Employee 3'])
```

Accessing the dataframe by index

```
print(data.iloc[0])
```

```
print(data.iloc[1])
```

```
print(data.iloc[2])
```

Add a new column

```
data['Job'] = ['Cook', 'N/A', 'Cashier']
```

```
print(data)
```

#	Name	Age	Job
Employee 1	Spongebob	30	Cook
Employee 2	Patrick	35	N/A
Employee 3	Squidwards	50	Cashier

Add a new row

```
new_row = pd.DataFrame({'Name': 'Sandy', 'Age': 28,  
                        'Job': 'Engineer'})
```

```
df = pd.concat([data, new_row]) # Index is not set up for this  
print(df) # dataframe
```

#	Name	Age	Job
Employee 1	Spongebob	30	Cook
Employee 2	Patrick	35	N/A
Employee 3	Squidwards	50	Cashier
0	Sandy	28	Engineer

Adding the index to the value

```
new_row = pd.DataFrame([{'Name': 'Sandy', 'Age': 28, 'Job': 'Engineer'}], index=['Employee 4'])
df = pd.concat([data, new_row])
print(df)
```

#	Name	Age	Job
Employee 1	Spongebob	30	Cook
Employee 2	Patrick	35	N/A
Employee 3	Squidwards	50	Cashier
Employee 4	Sandy	28	Engineer

More practice to add new rows

```
new_rows = ([{'Name': 'Sandy', 'Age': 28, 'Job': 'Engineer'},
              {'Name': 'Eugene', 'Age': 60, 'Job': 'Manager'}],
            index=['Employee 4', 'Employee 5'])
df = pd.concat([data, new_rows])
print(df)
```

Importing the Comma-Separated values (CSV) and Javascript Object Notation (JSON) files

```
df = pd.read_csv('data.csv', index_col=['Name'])
```

Use .read_csv for CSV files and .read_json for Json files.

In data.csv file we have the full data of 150 pokemons. => Attachment <

let we print the file

#	Name	No	Type 1	Type 2	Height	Weight	Legend
	Bulbasaur	1	Grass	Poison	0.7	6.9	0
	Ivysaur	2	Grass	Poison	1.0	13.0	0
	Venusaur	3	Grass	Poison	2.0	100.0	0
	Charmander	4	Fire	NaN	0.6	8.5	0
	Charmeleon	5	Fire	NaN	1.1	19.0	0
...	...	...	...	...	...	...	...
	Moltres	146	Fire	Flying	2.0	60.0	1
	Dratini	147	Dragon	NaN	1.8	3.3	0
	Dragonair	148	Dragon	NaN	4.0	16.5	0
	Dragonite	149	Dragon	Flying	2.2	210.0	0
	Mewtwo	150	Psychic	NaN	2.0	122.0	1

150 rows X 6 columns

Access using .loc function
 print(df.loc['Charizard': 'Blastoise', ['Height', 'Weight']])

#	Name	Height	Weight
	Charizard	1.7	90.5
	Squirtle	0.5	9.0
	Wartortle	1.0	22.5
	Blastoise	1.6	85.5

Print(df.iloc[0:150:2, 0:3])

#	Name	No	Type 1	Type 2
	Bulbasaur	1	Grass	Poison
	Venusaur	3	Grass	Poison

Charmeleon	5	Fire	NaN
Squirtle	7	Water	NaN
Blastoise	9	Water	NaN
...	...	...	...
Kabutops	141	Rock	Water
Snorlax	143	Normal	NaN
Zapdos	145	Electric	Flying
Dratini	147	Dragon	NaN
Dragonite	149	Dragon	Flying

[75 rows X 3 columns]

Selection by columns

```
print(df["Height"].to_string())
```

```
print(df["Weight"].to_string())
```

Multiple column selection

```
print(df[["Name", "Height", "Weight"]])
```

Check the Pokemon through input function

```
Pokemon = input("Enter a Pokemon Name: ")
```

try:

```
print(df.loc[Pokemon])
```

except KeyError:

```
print(f"{Pokemon} not found!")
```

Filtering: Keeping the rows that match a condition

```
tall_pokemon = df[df["Height"] >= 2]
```



```
Print (tall-pokemon)
```

```
legendary-pokemon = df[df["Legendary"] > 0]
Print (legendary-pokemon)
```

```
Water-pokemon = df[(df["Type1"] == "Water") | (df["Type2"] == "Water")]
Print (Water-pokemon)
```

```
bb-pokemon = df[(df["Type1"] == "Fire") &
                 (df["Type2"] == "Flying")]
Print (bb-pokemon)
```

Aggregate functions: Reduces a set of all values into a single summary value used to summarize and analyze the data. Often used with groupby() function.

```
Print(df.mean(numeric_only = True)) # for mean
```

# No	75.500000
Height	1.200000
Weight	46.231333
Legendary	0.026667

```
Print(df.min(numeric_only = True)) # for minimum
```

```
Print(df.max(numeric_only = True)) # for maximum
```

```
Print(df.sum(numeric_only = True)) # for Sum
```

```
Print(df.Count()) # for counting
```


Single Column

```
Print(df['Height'].mean())  
Print(df['Height'].sum())  
Print(df['Height'].min())  
Print(df['Height'].max())  
Print(df['Type2'].count())
```

Groupby(): Group all the elements of the column

```
group = df.groupby('Type1')  
Print(group['Height'].mean())  
Print(group['Height'].sum())  
Print(group['Height'].min())  
Print(group['Height'].max())  
Print(group['Height'].count())
```

Data Cleaning: The process of fixing/removing (=) Incorrect, Incomplete or irrelevant data. ~ 75% of work done with Pandas is data cleaning.

```
Cleaned = df.drop(columns=['Legendary', 'No'])  
Print(Cleaned)
```

2. Handle Missing Data

```
missing-data = df.dropna(subset=['Type2'])  
data = df.fillna({'Type2': 'None'})  
Print(data.to_string())
```

Fix inconsistent values

```
df['Type1'] = df['Type1'].replace({'Grass': 'GRASS', 'Fire': 'FIRE', 'Water': 'WATER'})
```



```
print(df.to_string())
```

Standardize text (4)

```
df = pd.read_csv('data.csv')  
df['Name'] = df['Name'].str.lower()  
print(df.to_string())
```

5. Fix Data Types

```
df['legendary'] = df['legendary'].astype(bool)  
print(df.to_string())
```

6. Remove duplicate data

have to add some duplicate values, at that time I'm not adding the duplicate values.

```
df = df.drop_duplicates()  
print(df.to_string())
```